Summary of Lecture on Pre-training 1/29

Yihang Zhu (yihangz), Yiqun Wang (wyq), Yicheng Tao (yctao), Xiangchen Song (xxxchen),

Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning

Problem and Motivation

Large-scale deep learning models (like GPT-3 with hundreds of billions of parameters) face severe engineering challenges in distributed training: developers need to manually design complex parallel strategies and fine-tune the combination of data parallelism, operator parallelism, and pipeline parallelism. This process highly depends on knowledge from both machine learning and systems, and is optimized toward specific models and cluster environments. This process is time- and energy-expensive, and blocks the capability of researchers to quickly explore new model architectures, thus becoming a bottleneck of large-scale deep learning development.

The importance of this problem is that the continuous growth of model scale is proven to bring fundamental improvement in capability, but the complexity of training these extremely large model systems is growing exponentially. Correctly optimized parallel strategies are proven by research to bring orders of magnitude improvement in performance, but existing systems either need to be designed by experts (like Megatron-LM), or automatic strategies cover limited space. Realizing fully automatic generation of parallel strategies can allow developers to efficiently train arbitrarily large models and lower the threshold of distributed training, therefore increasing the speed of frontier AI research and industrial applications.

Related Works

Traditional deep learning parallel strategies can be divided into three categories: data parallelism, operator parallelism, and pipeline parallelism. The most advanced manually designed systems, like Megatron-LM, adopt 3D parallelism, which assumes that the model is constructed based on repeated Transformer layers and designs all layers' parallel configurations uniformly. However, this type of design severely depends on the homogeneity assumption of models, cannot adapt to heterogeneous architectures like Wide-ResNet, and is hard to adapt to different cluster topologies.

Automatic parallel configuration research tries to relieve the burden of manual configuration, but has apparent limitations. Tofu uses dynamic programming to generate the best intra-operator parallel strategy for a single operator; FlexFlow proposes the SOAP formula and uses MCMC random search, but only supports device parallelism and not pipeline parallelism, and its search algorithm cannot expand to large-scale graphs or clusters; TensorOpt, although considering memory and computation cost, is still limited to a single parallel dimension. These systems either cannot combine multiple parallel technologies or have limited search space, missing opportunities for large performance optimization. They also fail to build layered optimization frameworks to handle the complex dependencies between different parallel strategies.

Recent research has started to re-categorize parallel technologies by dividing parallelism into intra-operator and inter-operator. Varuna focuses on automated pipeline and data parallel optimization on low-bandwidth clusters; Piper, although combining two different parallelisms, depends on manual design of the intra-operator strategy. Alpa is the first end-to-end system that can build a complete layered search space and use compiler technology to generate execution plans covering all parallel dimensions, without human interference.

Solution Overview

The key insight of Alpa is to organize parallel technologies into a layered space and match them to the layered communication structure of compute clusters. Intra-operator parallelism produces frequent group communication (like all-reduce) and can be mapped to device groups with high bandwidth (like GPUs in the same server). Inter-operator parallelism only communicates at the boundaries of phases, and most of it is point-to-point communication, and can be mapped to device groups with low-bandwidth connections (like GPUs on different servers). Based on this, Alpa expresses the parallel plan into a two-layer structure: first decide how to partition the cluster into a device mesh, then decide how to cut the model into phases and assign them to the mesh.

On the intra-operator layer, Alpa uses an uniform sharding strategy, turning the problem into an ILP. The system enumerates possible parallel algorithms for each operator in the computation graph, designs the sharding specification that describes the layout of tensors on the device mesh, and models the communication cost of resharding. The ILP solver minimizes the overall cost (both communication and computation) of the subgraph, and post-processing optimizations such as operator fusion further reduce the communication cost (for example, using reduce-scatter + all-gather to replace all-reduce).

On the inter-node layer, Alpa uses a dynamic programming algorithm to cut the model and cluster together. The DP state F(s, k, d; tmax) represents the minimum overall latency when cutting operators ok to oK into S phases and assigning them to d devices, with each phase's latency less than tmax. The algorithm enumerates possible phase cutting points and sub-mesh

shapes, repeatedly calls the intra-node compiler to obtain the actual execution latency for each phase–mesh pair, and uses operator clustering techniques to combine lightweight operators to reduce the complexity of the DP. The ultimate goal is to minimize the overall latency of the pipeline.

The Alpa compiler structure contains three key phases: intra-operator compilation generates phase-mesh assignment plans; inter-operator compilation generates sharding strategies inside the device mesh for each phase; runtime compilation handles cross-mesh communication and generates static execution instructions. This design allows Alpa to uniformly express all current parallel methods, support heterogeneous model structures and cluster configurations without human intervention, and generate approaches that can even surpass manual optimization.

Limitations

Cross-stage communication is not modeled in the optimizer.
Alpa's optimization algorithms "do not model the communication cost between different stages" (they argue it is usually small), because adding it into the DP/ILP would require enumerating far more stage–mesh combinations and DP states, potentially blowing up the search space.

Micro-batch count BB is a hyperparameter, not jointly optimized.
The inter-operator pass treats the number of micro-batches BB as a tunable knob; Alpa does not optimize it in the main formulation and instead suggests searching it by enumeration.

Pipeline scheduling is simplified (static, linear).
The inter-op pass models pipeline parallelism with a static linear schedule, and does not consider more dynamic schedules (e.g., exploiting parallel branches in the computation graph).

Limited handling of overlap and dynamic graphs/shapes.
Alpa does not optimize for the best compute/communication overlap scheme, and it can only handle static computational graphs where tensor shapes are known at compile time.

Compilation/search overhead can be large.
Even with optimizations (parallel compilation, XLA-instruction-level cost model), the full compilation + search can take several hours; much of the time is spent enumerating and profiling stage–mesh pairs.

Ecosystem / portability constraints.
The implementation is built with JAX as frontend and XLA as backend, so adopting Alpa's approach in other stacks typically requires non-trivial integration work.

Future Research Directions

Richer cost modeling without exponential blow-up
Extend the optimizer to incorporate cross-stage communication (and cross-mesh resharding effects) while keeping search tractable—e.g., hierarchical approximations, selective modeling of "heavy" boundaries, sampling-based estimation, or learned cost models.

Joint optimization of micro-batching and pipeline structure
Instead of enumerating BB, integrate BB (and possibly the number of stages) directly into the DP objective/constraints, or derive analytic bounds to prune the search space.

More general and dynamic scheduling
Move beyond static linear pipelines to schedules that handle branchy graphs, heterogeneous stages, and more advanced interleavings (while still guaranteeing correctness and good utilization).

Automatic compute/communication overlap optimization
Introduce an explicit scheduling layer that reasons about streams, asynchronous collectives, and overlap opportunities, and couples this with a more detailed runtime cost model.

Support for dynamic shapes / partially dynamic graphs
Add mechanisms like shape polymorphism, guarded/specialized compilation, caching, and runtime re-optimization so Alpa-like systems can handle more real-world workloads.

Faster compilation via reuse and incrementality
Use plan caching, warm-start ILP/DP, profiling reuse across similar model sizes, and incremental compilation to reduce "hours-level" planning overhead—especially important for rapid iteration and hyperparameter sweeps.

Broader backends and more realistic cluster settings
Generalize beyond a single frontend/backend stack (e.g., broader compiler IR support) and expand to heterogeneous devices/topologies, elastic/fault-tolerant training, and multi-tenant environments.

Summary of Class Discussion

Today's discussion focused on failure/bottleneck scenarios in distributed training, with Alpa as the main reference point. A key concept that came up was re-sharding cost: when one operator produces tensors with a certain sharding/replication pattern (e.g., fully replicated, row-wise sharded, column-wise sharded),

but the next operator requires a different layout. Aligning these layouts requires real GPU-to-GPU communication (e.g., all-gather, all-to-all, all-reduce, or other data rearrangements), which can become a major source of overhead.

This is directly relevant to Alpa because Alpa's automated parallelization relies on selecting sharding strategies across operators and minimizing costly re-sharding between consecutive operators. The discussion emphasized that as long as you use some form of tensor/operator parallelism, you inevitably deal with sharding decisions and the associated communication, and communication costs can explode if heavy collectives frequently cross slower interconnects (e.g., across nodes). This motivated the intuition that expensive neighboring operations should ideally be placed and executed in a way that avoids unnecessary communication boundaries.

Another theme was the question of whether there are "known best" manual parallelization patterns that could be directly applied (e.g., standard pipeline stage splits). The response highlighted that Alpa is fundamentally cost-model/solver-driven: you provide a search space and the system explores it to optimize performance. Integrating human rules into such a solver is not straightforward; a more plausible approach would be adding a separate pass that prunes the search space using heuristic rules, then running the cost-based search on the reduced space.

Finally, the discussion noted a practical barrier: Alpa is a Google system implemented in the JAX/XLA ecosystem, which can make reproduction and comparison harder in environments that are primarily PyTorch-based, even if the core ideas are broadly applicable.

Summary of FSMoE: A Flexible and Scalable Training System for Sparse Mixture-of-Experts Models

Problem and Motivation

Mixture-of-Experts (MoE) models have become a key technique for scaling large language models (LLMs) by increasing parameter counts while keeping per-token computation sublinear. By activating only a small subset of experts per token, MoE models enable trillion-parameter architectures that would otherwise be computationally infeasible. However, this sparsity introduces complex system-level challenges, particularly in distributed training environments.

From a systems perspective, MoE training involves multiple tightly coupled components: token routing, data layout transformation, expert computation, and extensive collective communication. Modern large-scale MoE training typically combines several parallelism paradigms simultaneously, including data parallelism (DP), model parallelism (MP), expert parallelism (EP), and expert-sharding parallelism (ESP). While these paradigms allow MoE models to scale across GPU clusters, they also introduce heavy communication overhead, especially from All-to-All collectives and gradient synchronization.

Existing MoE systems such as DeepSpeed-MoE and Tutel incorporate various optimizations, but they suffer from two fundamental limitations. First, they are inflexible: most systems hard-code assumptions about routing functions, data layouts, or communication strategies, making it difficult to experiment with new MoE designs. Second, they are often sub-optimal in scheduling, particularly under hybrid parallelism. Prior systems mainly focus on overlapping expert computation with All-to-All communication, while ignoring the heterogeneous nature of communication (intra-node vs. inter-node) and the interaction between MoE communication and gradient synchronization.

Empirical evidence in the paper shows that communication can account for more than 50% of MoE training time on modern GPU clusters. This motivates the need for a training system that is both flexible enough to support diverse MoE designs and intelligent enough to co-schedule computation and communication near-optimally across different hardware configurations.

Related Works

Prior work on MoE training optimization spans three main directions: MoE algorithm design, communication algorithm optimization, and system-level scheduling.

On the system side, Tutel and DeepSpeed-MoE are widely used MoE training frameworks. Tutel introduces adaptive pipelining to overlap All-to-All communication with expert computation, while DeepSpeed-MoE integrates MoE support into a general large-model training framework. However, both systems rely on heuristics or limited search spaces for scheduling and provide restricted support for routing functions and data layouts.

FastMoE, FasterMoE, and PipeMoE further explore communication–computation overlap, with PipeMoE proposing an adaptive partitioning of input tokens to improve overlap efficiency. Lina addresses network contention in backpropagation by partitioning gradients into fixed-size chunks. Despite these advances, existing approaches typically optimize only one dimension of the problem (e.g., All-to-All overlap) and do not jointly consider inter-node communication, intra-node communication, expert computation, and gradient synchronization.

Beyond MoE-specific systems, recent work on fine-grained overlap and kernel fusion (e.g., FLUX, CoCoNet, T3) demonstrates that deeper integration of computation and communication can significantly improve performance. However, these approaches are not designed specifically for the modular and heterogeneous structure of MoE layers.

FSMoE positions itself as a unifying system that integrates ideas from these prior works while addressing their limitations through modular abstraction and model-driven scheduling.

Solution Overview

The key idea of FSMoE is to decouple MoE flexibility from scheduling efficiency and then re-optimize execution globally. The system introduces three core design contributions.

First, FSMoE provides a unified modular abstraction for MoE layers. Each MoE layer is decomposed into six sub-modules: Gate, Order, I-Order, Dispatch, Combine, and Expert. This design allows different routing functions, data layouts, and communication algorithms to be plugged in without invasive code changes. FSMoE implements several popular gating mechanisms (e.g., GShard, Sigmoid, X-MoE, Expert Choice) and multiple All-to-All algorithms, enabling easy experimentation with new MoE variants.

Second, FSMoE introduces a generic scheduler with online profiling. Instead of relying on static heuristics, FSMoE profiles the execution time of each sub-module and builds linear performance models for both communication and computation. Using these models, the scheduler determines how to pipeline tasks to minimize total execution time. Crucially, FSMoE explicitly distinguishes between intra-node communication (e.g., ESP AllGather/ReduceScatter via NVLink) and

inter-node communication (e.g., EP All-to-All via InfiniBand), allowing the system to exploit hardware asymmetry.

Third, FSMoE proposes adaptive scheduling for both forward and backward passes, including a novel gradient partitioning strategy. The system recognizes that the optimal pipeline degree differs between forward and backward propagation due to additional gradient computations and synchronization. FSMoE models these differences and selects separate pipeline configurations for each phase. In backpropagation, FSMoE adaptively partitions gradients so that Gradient-AllReduce can be overlapped with MoE communication and computation, rather than being treated as a blocking operation.

Together, these techniques enable FSMoE to co-schedule expert computation, intra-node communication, inter-node communication, and gradient synchronization in a near-optimal manner.

Limitations

Despite its strong performance, FSMoE has several limitations. First, the system relies on accurate performance modeling, which requires micro-benchmarking on each new hardware platform. While this cost is amortized over training, it may be inconvenient in highly dynamic or heterogeneous environments.

Second, FSMoE's scheduling assumes common but specific configurations, such as aligning MP and ESP within a node. While this matches many real-world deployments, the benefits may be reduced in less structured or highly asymmetric cluster topologies.

Third, FSMoE focuses primarily on training performance. The paper does not explore inference-time implications, memory fragmentation issues, or fault tolerance, which are important considerations for production MoE systems.

Finally, the system introduces additional complexity compared to simpler frameworks, which may increase engineering overhead for adoption and debugging.

Future Research Directions

FSMoE opens several promising avenues for future research. One direction is extending the scheduler to support dynamic or adaptive routing strategies where token distributions change

significantly during training. Another is integrating FSMoE with kernel fusion or hardware-software co-design techniques to further reduce overhead.

Summary of Class Discussion

Q1: Why does backpropagation (especially Gradient-AllReduce) happen after MoE communication instead of earlier?

A: Because Gradient-AllReduce is a heavy inter-node communication, the system delays it to overlap with other operations when possible. In some schedules (e.g., case 4), Gradient-AllReduce appears after AlltoAll-Combine, but without careful scheduling it may not overlap with other tasks, causing inefficiency. FSMoE addresses this by partitioning gradients to enable overlap.

Q2: In the slides' four scheduling cases, why does case 4 appear less often in practice?

A: Case 4 depends on specific system configurations and constraints, so it is not universally applicable. As noted, Table 4 in the paper lists configurations where different cases apply. Not all hardware or parallelism setups satisfy the assumptions required for case 4.

Q3: How do FSMoE and Alpa differ, and can their ideas be combined?

A:

- Alpa focuses on automatic parallelization and planning, largely ignoring detailed communication modeling.
- FSMoE focuses on fine-grained communication scheduling, especially for MoE workloads.

The professor suggested a potential research direction: combining Alpa's planning/compilation with FSMoE's communication-aware scheduling, especially for heterogeneous clusters.

Q4: How does heterogeneity affect optimization in systems like Alpa?

A: In Alpa, heterogeneity is modeled via parameters (e.g., communication cost variables). If inter-node communication cost is zero, nodes are treated as homogeneous; otherwise, heterogeneity is reflected in the optimization formula. However, this increases complexity and planning time.

Q5: Beyond scaling, what new challenges do MoE architectures introduce?

A: MoE introduces:

- New parallelism types (EP, ESP)
- A much larger search space
- The need for composable system designs

This makes fully automated optimization harder and motivates hybrid approaches.

Q6: Should parallelization be fully automated or partially manual?

A: Pure automation (like Alpa) can be slow and unpredictable. In practice, hybrid strategies work best:

- Users specify constraints or hints
- Systems automatically optimize within those constraints

FSMoE fits this philosophy better than fully automated planners.

Q7: How is planning handled in real-world big data systems?

A: Most systems use hybrid planning:

- Rule-based heuristics + cost models
- Plans are usually fixed at runtime
- If failures occur, systems may switch to fallback plans rather than recomputing everything

Dynamic replanning without stopping execution remains a hard open problem.

Q8: What is sharding, and is Alpa's sharding applicable to FSMoE?

A: Sharding means splitting computation or parameters across GPUs. Alpa and FSMoE use similar ideas, but FSMoE emphasizes that tensor-parallel operations must stay within fast interconnects (e.g., NVLink); otherwise, communication costs dominate.

Q9: Is Alpa outdated compared to newer manual parallelization approaches?

A: Alpa (2022) is considered static and older. Newer approaches allow more manual control, including merging operations on the same GPU to avoid communication entirely.

Q10: Can Alpa's static design be extended to dynamic settings?

A: This was raised as an open project question. While Alpa is designed for static execution plans, some components might be reusable in dynamic or adaptive systems—but this remains an open research direction.